Blockchain-Assisted Zero-Trust Access Control for Industrial IoT: Edge Enforcement, Tamper-Evident Audit and Cloud Evaluation

Syed Irfan Raza Naqvi[a,*], Muhammad Umar Aftab[a]

[a] *FAST School of Computing, Department of Computer Science, National University of Computer and Emerging Sciences (FAST-NUCES), Lahore, Pakistan*

* Corresponding author. E-mail: l259000@lhr.nu.edu.pk; irfanrazanaqvi9@gmail.com. Co-author: umar.aftab@nu.edu.pk

Abstract

Zero-trust access control for industrial IoT (IIoT) means checking every request, yet putting a blockchain on the authentication path adds seconds of delay and exhausts ledger capacity. We describe BC-ZTA, an architecture in which edge enforcement points decide from cached ledger state that they verify against a Merkle root, while identity, trust and audit updates reach a permissioned Hyperledger Besu network asynchronously. We built it, together with six baselines (among them Keycloak, OpenLDAP and a central directory with pushed revocation), and tested it on seven cloud VMs with four validators. None of 1750 forged, replayed, stale, unregistered, tampered or policy-violating requests was accepted, though these are negative tests, not attacks on a compromised gateway or administrator. A revoked device was refused after 1.3 s on average, and the enforcement point stops granting access once its state source has been lost for longer than twice the cache lifetime. Decisions took 5.4 ms at 50 devices, against 49.1 ms for a central directory behind an injected 40 ms delay; that gap is one wide-area round trip, not a general speed-up, and a central directory with pushed revocation matches the latency and revokes faster (109 ms). What the ledger adds is distributed trust and audit that third parties can verify: in local tests, anchoring exposed five kinds of after-the-fact edit by an operator, one of which a hash-chained log misses, and a Merkle proof stays below 700 bytes and 0.5 ms at one million devices. A consensus halt delays revocation by minutes. Devices were emulated, all machines sat in one cloud zone, and trust scoring and micro-segmentation are design-level only.

Keywords: Zero trust; Access control; Industrial IoT; Blockchain; Audit integrity; Revocation; Smart contracts

1. Introduction

1.1. Background and Problem

Industrial IoT (IIoT) plants link PLCs, sensors, actuators, edge gateways and cloud analytics. Their operational-technology networks used to depend on a hardened perimeter and on trust between devices on the same segment. Remote maintenance and the merging of IT and OT have removed that perimeter, and one compromised vendor laptop or sensor can now be the starting point of lateral movement [1, 2]. Zero trust, as set out in NIST SP 800-207 [3], replaces location-based trust with verification of every request, and recent reviews show it spreading across application domains [4–6].

IIoT makes this hard for four reasons. Devices are constrained and cannot run heavy cryptography or monitoring agents. Protocols and vendors differ, which complicates uniform identity management [7, 8]. Control loops leave only milliseconds for a security decision. And thousands of devices join, rotate keys and are revoked all the time. A central identity or policy server is then a single point of failure and

an attractive target, and logs kept by the same operator cannot be trusted independently [5]. Following NIST and the critiques of the model [9], we need per-device cryptographic identity (**R1**), authorisation of every request by a policy decision point (PDP) enforced at a policy enforcement point (PEP) (**R2**), least-privilege short-lived sessions (**R3**), continuous trust assessment [10] (**R4**), micro-segmentation (**R5**) and tamper-evident audit (**R6**), with R1–R5 fitting inside control-loop latency budgets. The evaluation covers these unevenly: R1, R2 and R6 are exercised end to end, R3 only through contract unit tests and session lifetimes, while R4 (trust scores are voted values, not learned) and R5 (no segmentation was deployed) are design-level elements that we do not evaluate.

1.2. Why a Ledger, and What Is Missing

A permissioned blockchain can hold identity, policy and reputation state and an audit trail that plant, vendor and integrator domains share without trusting any single one of them. It has been applied to zero-trust IoT access [11, 12], cross-domain digital-twin access [13], IIoT data protection [14] and contract-based access control [15, 16]. Three gaps remain. Many designs put the ledger on the request path, and the cost shows: one Ethereum-based zero-trust framework reports latency rising from 49.33 ms to 74.0 ms and throughput dropping from 50 to 30.77 req/s [17]. Scaling of authentication and authorisation workloads is rarely evaluated, apart from sharding-based work [18], because most studies report accuracy or encryption cost [19, 20]. And continuous tamper-proof audit, named repeatedly as an open problem [5], becomes expensive when every event is written on-chain.

1.3. Contributions

We design a blockchain-assisted zero-trust architecture (BC-ZTA) whose decisions do not wait for consensus, bound how stale its cached state can become, and make audit cost grow more slowly than the request rate. Specifically:

- **Architecture.** A three-tier design in which the PEP/PDP decide from edge-cached, Merkle-verified ledger state, so the blockchain stays *off* the request path (Section 3).
- **Smart contracts.** Four Solidity contracts (identity registry with epoch commitments, quorum-based trust reputation, ABAC access control, audit log with Merkle batch anchoring) with 17 passing unit tests.
- **Prototype and measurements.** A running PEP/PDP service and load generator, six baselines (central directory, local ABAC, synchronous blockchain, real Keycloak and OpenLDAP with and without a PEP cache, and a central directory with pushed revocations), evaluated on seven Google Cloud VMs with a four-validator Hyperledger Besu IBFT 2.0 network: latency and goodput up to 200 devices, revocation delay, security tests, validator failure, central-service outage, ledger flooding, loss of the state source, a consensus halt and PEP scale-out, and gas; local tests cover independent verification of tampered audit batches and Merkle proof cost up to one million devices (Sections 5–6).
- **Security analysis and its limits.** An argument for each security goal under explicit assumptions, together with the residual risks we found: replay across gateways, trust in a single ledger node and incomplete audit (Section 4.6). A measured comparison shows where the ledger helps and where it does not (Section 6.8).
- **Bounded staleness.** Event-driven invalidation reduces the mean revocation delay from 6.6 s to 1.3 s (Section 6.6).

Section 2 reviews related work, Section 3 presents the architecture, Section 4 the use case and threat model, Section 5 the methodology, Section 6 the results and Section 7 concludes.

2. Related Work

2.1. Blockchain-Based Security for IIoT and IoT

Ledger-based security work for IIoT and IoT falls roughly into data protection, access control and identity. On the data side, Essaid and Ju combine blockchain, edge computing and smart contracts for integrity and threat detection [14]. For access control, Hyper-IIoT offers a contract-inspired scheme for constrained devices [15], Guo applies model checking to IoT access-control contracts [16], and Zhai et al. authenticate cross-domain IIoT devices with Chebyshev chaotic maps [21]. Identity schemes include epoch-based batch registration with sparse Merkle trees, tested up to 7,000 devices on 20 fog nodes with under 5 MB of on-chain storage [8]. Two recent IIoT designs make large identity sets verifiable: Ding et al. prove identifier-to-key bindings in a DID registry with a sliding-window accumulator and an on-chain/off-chain query strategy [22], and Xu et al. store DID documents of manufacturing entities in a Merkle structure on Hyperledger Fabric [23]. They address the verifiability and query cost of the identity registry; BC-ZTA uses an epoch Merkle commitment for the same purpose, and additionally measures per-request enforcement and revocation against production IAM baselines. DIDs on IOTA were also studied, where generating an identifier loaded a Raspberry Pi far more than a PC [24]. For scale, DAG ledgers have been proposed for intelligent buildings [25], and a smart-city framework with a lightweight adaptive proof-of-stake reports a threefold throughput gain and 30% lower latency than conventional blockchains [26].

2.2. Zero Trust for IIoT

Zero-trust designs for industrial networks mostly come without a ledger. Federici et al. built a zero-trust remote-access architecture for IIoT [1], and Zanasi et al. combine micro-segmentation, SDN and a central policy layer [2]. Ma and Chiu add a risk-based access-control engine that rates a device by the behaviour of its neighbours [10], Mthethwa et al. address non-human identities in IoT [7], Wan et al. combine PUF-based device authentication with continuous location checks during a session [27], and Panahi's ZT-IIoT scores every cross-layer transaction (a Modbus read, an OPC UA subscription, an MES order) with an anomaly-detection trust module and reports a maximum enforcement delay of 2.7 ms against a 10 ms target in simulation, on a hardware testbed and on the SWaT dataset [28]; its metric is not the same as our decision latency, but it shows the budget such designs aim for. Network-side micro-segmentation, such as a three-tier framework that reports about 130 ms latency for 1,000 simultaneous users [29], limits lateral movement inside the network and is complementary to the decision and revocation path studied here. Broader overviews include a survey of zero-trust architecture [30], zero-trust foundation models for IoT [31] and the applications and open problems of zero trust [32].

2.3. Blockchain-Assisted Zero Trust

Combining the two is more recent. Aleisa pairs a blockchain-based ZTA with post-quantum and zero-knowledge techniques and reports 98% privacy preservation [11]. Nie et al. use smart contracts with inner-product encryption for 6G IoT access control and report 14% better encryption efficiency and 18% lower access latency [12]. Sharma et al. add federated transfer learning for Industry 5.0 [19], and Mishra et al. combine a zero-trust digital twin, intrusion detection and lightweight blockchain auditing [20]. Salim et al. reach 2,500 TPS at 700 nodes with sharding driven by reinforcement learning [18]. Other variants target FinTech [17], clinical trials [33] and FPGA supply chains [34].

2.4. Alternatives That Need No Blockchain

Much of what a ledger does here can be done without one, so any evaluation should ask how much. On the web, revocation is handled by CRLs [35] and OCSP [36], and a server can staple a signed status answer to the TLS handshake [37]; a central authority that pushes signed revocations to edge enforcement points is therefore an obvious rival to ledger-driven invalidation. Transparency logs such as Certificate Transparency [38] give audit that third parties can check through log operators and monitors, with no consensus protocol. Tamper-evident logging does not need a blockchain: a recent preprint on IoT edge logs commits batches to Merkle trees and reports that corrupted entries were always detected, with proofs of about 1 KB and about 22 ms per entry to verify [39]. Anchoring Merkle roots on a ledger uses the same idea with a different publication medium. Decentralised identifiers [40] separate control of an identifier from any registry operator. Blockchain-based access control for IoT puts the policy decision or the policy store on the ledger, from FairAccess [41] to scalable access-management architectures [42]. Among permissioned ledgers, Hyperledger Fabric uses execute-order-validate [43], whereas Istanbul BFT [44], which we use, brings classical Byzantine agreement [45] to an EVM chain. Finally, an Internet-wide study found many OPC UA deployments with missing or weak authentication [46], which is why we insist on per-device cryptographic identity (R1). We therefore compare against central directories, real Keycloak and OpenLDAP services and a central directory with pushed revocation.

2.5. Comparison and Gap

Table 1 compares representative works on four criteria, taken from what each paper reports.

TABLE 1. COMPARISON OF RELATED WORK

Work	Blockchain	Zero trust	IIoT focus	Scale eval.
[18]	Yes	Yes	No	Yes
[11]	Yes	Yes	No	No
[12]	Yes	Yes	No	Part
[19]	Yes	Yes	Part	No
[20]	Yes	Yes	Yes	Part
[13]	Yes	Part	Yes	Part
[15]	Yes	No	Yes	No
[8]	Yes	No	Yes	Yes
[10]	No	Yes	No	No
[2]	No	Yes	Yes	No
[22]	Yes	No	Yes	No
[23]	Yes	No	Yes	No
[27]	No	Yes	Yes	No
[28]	No	Yes	Yes	No
[29]	No	Yes	No	Yes
This work	Yes	Yes	Yes	Part

Yes = addressed; Part = partly; No = not addressed. Authors' reading of each paper's reported content; "Scale eval." means device or user scaling is reported; for this work it is partial (200 devices end to end, one million in a proof microbenchmark). Ratings of the five newest works rest on the published abstracts and summaries, so "No" there may mean "not stated".

Most blockchain-based zero-trust designs target generic IoT rather than IIoT, and only three of the listed works report device or user scaling [8, 18, 29]. In our (non-systematic) survey no work does all three of the following: keep the ledger off the request path, quantify how stale the cached state becomes and how to bound it, and cut audit cost by Merkle anchoring for an IIoT workload. That combination is what this paper evaluates.

3. Proposed Architecture

3.1. Design Objectives

D1 decisions from local state in milliseconds; **D2** no single party that can change identity records, policy or logs without detection (a plant authority still enrols devices, Section 4.2); **D3** bounded staleness of cached state; **D4** write load that grows slower than the request rate; **D5** a device-side workload of one ECDSA signature per request and one stored Merkle path (device-side cost was not measured, Section 6.16).

3.2. Architecture Overview

Fig. 1 shows three tiers. Devices form Layer I (IIoT and edge). Layer II is the zero-trust security layer at the edge gateway, hosting the PEP, PDP and trust engine. Layer III is the permissioned blockchain, whose identity, trust, access-control and audit services are implemented as smart contracts. Gateways cache Layer III state, so Layer II never waits for consensus.

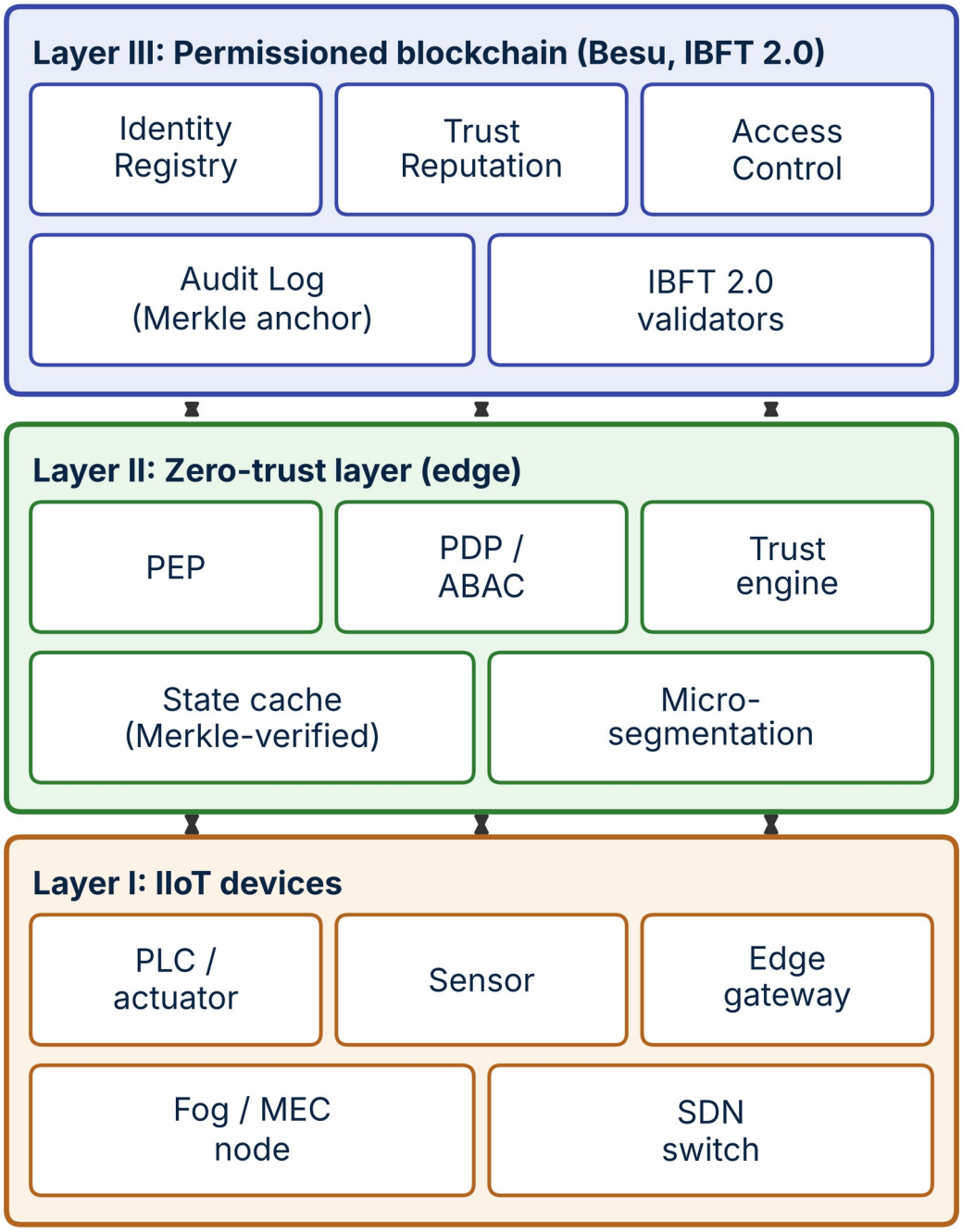


Fig. 1. Proposed BC-ZTA architecture: devices, edge/ZT layer, blockchain layer and contract services.

3.3. Edge and Zero-Trust Layers

Sensors, actuators and PLCs hold an ECDSA key pair bound to a DID and sign every request with a timestamp and a nonce. Edge gateways terminate sessions, keep a local cache of identity, trust and policy state, and forward anomaly telemetry to the trust engine. On each gateway the PEP intercepts every request. The PDP verifies the signature, checks the DID against a cached, Merkle-verified registry entry, fetches the trust score and policy, and returns allow or deny. Micro-segmentation permits only flows that hold a valid, unexpired session [2], and the trust engine turns behavioural features (rate, protocol conformance, firmware attestation) into score proposals.

3.4. Blockchain Layer

A permissioned Hyperledger Besu network with IBFT 2.0 consensus is operated by plant, integrator and vendor validators; with n validators it tolerates $f = \lfloor (n-1)/3 \rfloor$ Byzantine nodes. Gateways are read-only clients that subscribe to events. Only authorised registrars, validators and the policy administrator send transactions.

3.5. Identity, Trust and Access Control

Each device is a DID with a public-key hash and a device class. A plant-authorised registrar submits registrations, which can be grouped into *epochs*: the registrar commits the Merkle root of a batch, and a device later proves membership with a logarithmic-size path, an approach inspired by epoch-based IIoT identity schemes [8]. Key rotation and revocation are single transactions that emit events. Trust scores lie in [0,100] and start at 70. A score changes only when a quorum of validators proposes the same value, updates are rate-limited, and an anomaly flag subtracts 20 points at once (floored at 0) until the device is re-evaluated. Policies are attribute-based: for each resource the administrator sets the allowed device-class range, a minimum trust score and a session TTL. A request is granted if the DID is active, the class is in range and the score meets the threshold, and the grant yields a session identifier that expires after the TTL or on explicit revocation.

3.6. Smart Contracts

IdentityRegistry handles registration, key update, revocation, epoch commitment and membership checks. *TrustReputation* handles quorum voting and the anomaly flag. *AccessControl* holds policies, evaluates requests and grants or revokes sessions. *AuditLog* stores per-event logs and Merkle batch anchors and verifies inclusion. Source code and tests are in the public repository.

3.7. Continuous Verification

Fig. 2 summarises the workflow. Verification does not end at login: every request is re-evaluated against current cached trust, sessions are short-lived, and the gateway refreshes cached entries every *TTL* seconds or immediately on a *DIDRevoked* or *ScoreUpdated* event. Audit records are hashed into a batch whose Merkle root is anchored on-chain asynchronously (step 7), so logging never delays the response.

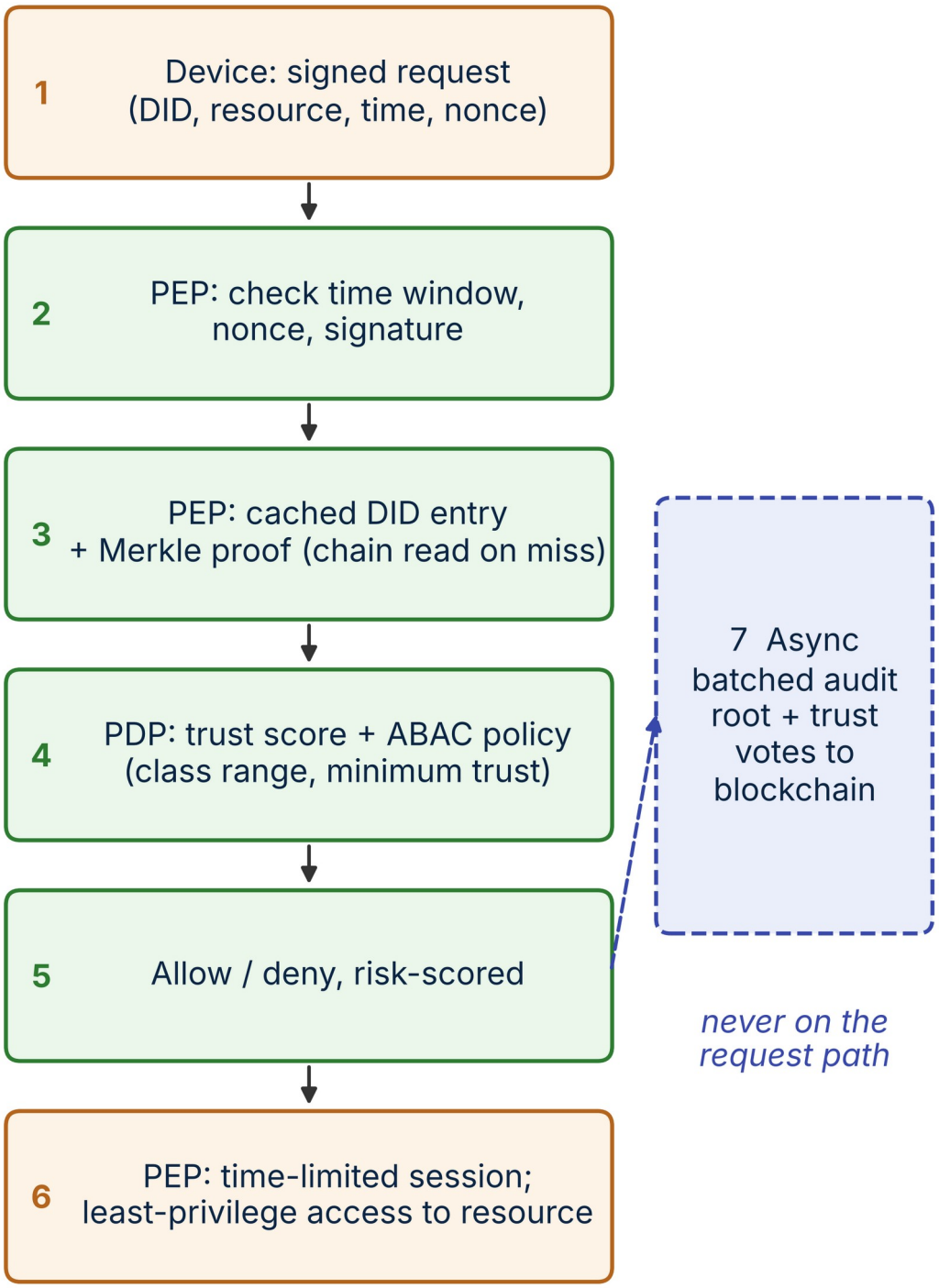


Fig. 2. Per-request verification workflow with cached state reads and asynchronous audit anchoring (step 7).

A request is r = (DID, resource, operation, t, n, σ): a timestamp, a nonce and the device's ECDSA signature over the other fields. The PEP accepts it only if (i) $|\text{now} - t| \le \Delta$ and n is new within Δ, which bounds replay; (ii) the DID is active in the cached registry, shown by a Merkle path against the latest

committed epoch root or by a direct chain read on a miss; (iii) σ verifies under the key whose hash is registered; and (iv) the PDP finds the trust score and policy satisfied. Otherwise the request is denied and an anomaly event is queued. The gateway also follows a fail-closed rule. If it cannot read the ledger, it keeps using a cached entry past its TTL for at most $k \cdot TTL$ after the entry was last read ($k = 2$ by default, parameter STALE_FACTOR) and then denies with reason "unavailable" until the ledger is back; $k = 1$ is the strict setting. A circuit breaker spares every request the read timeout while the source is down. The first cloud experiments (Sections 6.1–6.9) used an earlier prototype that denied as soon as a failed read followed TTL expiry ($k = 1$), and the $k = 2$ rule is evaluated in Section 6.10.

3.8. Ledger Access, Revocation and Read Quorum

A Merkle membership proof against an epoch root shows only that a DID was enrolled in that epoch. It stays valid after revocation, so it says nothing about freshness. Revocation is therefore enforced through the registry record (a revoked flag), which the gateway reads on a cache miss and which the *DIDRevoked* event invalidates. A gateway also trusts the Besu node it reads from, and a compromised node could return stale or forged state. A gateway without a validator of its own can instead read from 1 nodes pinned to the same block height and accept a state only if all answers agree: any $f + 1$ matching answers include an honest node, and IBFT 2.0 has instant finality, so there are no forks to reconcile. The prototype implements this option (QUORUM = 2 for $n = 4$); the default reads from one node, and the cost of the quorum is reported in Section 6.10.

4. Use Case and Threat Model

4.1. Use Case

A smart factory has several production cells, each with PLCs and sensors behind one edge gateway; a remote vendor engineer needs time-limited maintenance access to one PLC class; a SCADA historian consumes sensor data. Cells, vendor and integrator are separate administrative domains that jointly operate validators.

4.2. Assets and Trust Assumptions

Assets are device identities, policy and trust state, session tokens and audit records. We assume a majority-honest validator set (at most f Byzantine), secure device key storage, and gateways that are trusted for enforcement but may be compromised individually.

4.3. Adversary Model

A Dolev–Yao network adversary can eavesdrop, replay and inject messages; may compromise a subset of devices (including their keys) and one gateway; and may include an insider with administrative credentials in one domain but not in a validator quorum.

4.4. Threats and Countermeasures

Table 2 maps threats to mechanisms. It shows what the design intends; entries marked in the table are not evaluated end to end, and Section 6.6 states what the cloud tests cover.

TABLE 2. THREATS AND COUNTERMEASURES

Threat	Attack	Countermeasure
Spoofing	Forged device identity	DID + ECDSA signature; Merkle-verified registry entry
Replay	Re-sent request	Timestamp + nonce window at PEP; session TTL
Lateral movement	Compromised device probes peers	Per-resource ABAC, short sessions (micro-segmentation: design only, not implemented)
Insider / tamper	Admin edits policy or logs	On-chain policy and Merkle-anchored audit; M-of-N admin possible (unit tests, Section 6.13)
Trust poisoning	False score votes	Quorum votes, rate limit; non-validators rejected (contract unit tests only)
Stale state	Use of revoked identity	Event-driven cache invalidation, bounded TTL
Ledger DoS	Flooding the chain	Off-path reads; batched writes; non-validators cannot write trust

4.5. Security Goals

Authenticity of devices, least-privilege authorisation, revocation within a bounded delay, tamper-evident audit of anchored batches (attribution of a record to a signed request needs the v2 record format, which was tested locally only), and availability of access decisions during ledger congestion.

4.6. Security Analysis

We argue each goal under the assumptions of Sections 4.2–4.3 and four more: ECDSA over secp256k1 is unforgeable under chosen-message attack, Keccak-256 is collision- and second-preimage-resistant, gateway and device clocks differ by less than $\Delta/2$, and the enrolment authority registers only genuine devices. These are proof sketches, not machine-checked proofs; a ProVerif [47] model of the request protocol remains future work.

S1 Authenticity. A request passes only if the signature recovers an address whose hash equals the key hash registered for the claimed DID. A forger without the device key must break signature unforgeability, and registering a new key needs the authority (*updateKey*, *registerDID*). **S2 Replay.** A replayed request fails if it is older than Δ or its nonce was already seen by *the same gateway* within Δ. The signed message carries no gateway identifier, so the same request can be replayed once at a different gateway within Δ unless nonce caches are shared or the gateway identity is added to the signed fields; we neither implemented nor tested this mitigation. **S3 Revocation.** Once the revocation transaction is final (IBFT 2.0 finality is instant), a gateway that sees the event stops accepting the device within the polling interval, and a gateway that misses it enforces the revocation when its cache entry expires, i.e. within *TTL*. If consensus halts, the transaction cannot be finalised, so the bound runs from finalisation and not from submission (Section 6.10). **S4 Audit integrity.** Every batch root is anchored on-chain, so a stored record that is altered later no longer verifies, and changing an anchored root needs $f + 1$ validators. Completeness is not guaranteed: a compromised gateway can drop records before they are anchored (and up to the 1 s anchoring interval is exposed on a crash), and the measured prototype's *AuditLog* accepted anchors from any account (the released code restricts this to authorised gateways). **S5 Availability.** Decisions never wait for consensus and continue from cache for at most $k \cdot TTL$ when the ledger is unreachable (Section 3.7). A gateway whose node is compromised can deny service or serve stale state unless the read quorum of Section 3.8 is used. **Gateway compromise.** A compromised

gateway can approve any request and anchor batches that look valid: the ledger records what the gateway reports, not what happened, and the read quorum limits the state a gateway can be fed, not what it decides. A gateway that anchors two conflicting roots for the same batch leaves both on the ledger, so the conflict is visible (Section 6.13). **Staleness window.** If an attacker delays or blocks ledger events to a gateway, a revoked device keeps working until its cache entry expires, at most *TTL*. If the attacker also controls the node the gateway reads from, that node can keep serving an old state with a proof that still verifies, and the bound no longer holds unless the $f + 1$ read quorum is used; a stale answer then needs $f + 1$ colluding nodes. We derive these bounds by argument and did not run an eclipse or delay attack. **Administrator key.** In the measured prototype one policy-admin key could change policy and revoke sessions. The released code lets the admin role be handed to an M-of-N multisig contract (tested in unit tests, not measured), which removes the single key but raises the question of who the owners are. **Out of scope:** compromise of the enrolment authority, of $f + 1$ validators or of a device key before revocation, and traffic analysis.

5. Implementation and Experimental Methodology

5.1. Besu Blockchain Network

We deployed a four-validator Hyperledger Besu v26.9.0 IBFT 2.0 network ($f = 1$, block period 2 s, chain ID 2026, zero base fee, London EVM) in Docker on a GitHub-hosted 2-vCPU Linux runner for the contract-level micro-benchmark. All validators share that host, so network delay is negligible and CPU is contended. The end-to-end experiments ran separately on seven Google Cloud VMs (Section 5.6). A fresh network was started for each of 3 micro-benchmark repetitions, with contracts compiled by Solidity 0.8.20 (optimizer 200 runs, EVM "london"). Five scenarios used pre-signed transactions from 8 worker wallets: S1 sequential write confirmation, S2 read latency (500 *eth_call* evaluations of *AccessControl.evaluateRequest*), S3 per-event logging at 10–200 tx/s offered for 20 s, S4 batch anchoring at 1 tx/s, and S5 revocation visibility. Blocks were polled every 25 ms. Scripts, contracts and raw results are in the public repository listed under Data Availability.

5.2. Smart Contracts

The four contracts total about 370 lines. A Hardhat/Chai suite of 17 tests passes and covers registration, duplicate and unauthorised registration, revocation, key update, quorum scoring, score floor, rate limiting, non-validator rejection, access grant/deny, session expiry, epoch Merkle membership and audit batch inclusion proofs.

5.3. End-to-End Prototype and Load Generator

Besides the contract tests, we built the whole request path as running software (Node.js 20): a PEP/PDP service, a load generator and the baselines of Section 5.4. Every device owns a secp256k1 key and signs *DID* | *resource* | *timestamp* | *nonce*; its key hash and class are registered on-chain and its DID belongs to one Merkle epoch commitment. The PEP rejects timestamps older than 5 s and repeated nonces, recovers the signer, compares it with the registered key hash, checks the Merkle proof against the cached epoch root and evaluates the ABAC policy (device class 0–1, trust score ≥ 50). In BC-ZTA, identity, trust and policy state are read from Besu with *eth_call* on a cache miss and cached for 30 s. A block-polling subscription (20 ms) invalidates an entry when a *DIDRevoked*, *KeyUpdated* or *ScoreUpdated* event is mined, every decision is hashed, and once per second the Merkle root of the pending batch is anchored with *AuditLog.anchorBatch*. The load generator emulates N = 10, 25, 50, 100, 200 devices

that send one request per second with staggered phases, after 5 s of warm-up and over a 20 s measurement window; requests are signed beforehand so the generator is not the bottleneck. PEP, directory server, load generator and the four validators run on separate machines (Section 5.6).

5.4. Baselines

(B1) *Centralised PKI-IAM*: one lookup per request at a directory server on its own VM, whose responses are delayed by 40 ms with Linux *tc netem* to model a remote central service (about 50 ms per lookup in total, Section 6.1). (B2) *ABAC without blockchain*: a local attribute store at the PEP. (B3) *Blockchain-only (sync)*: for every request the PEP reads state with *eth_call*, sends an *AuditLog.logEvent* transaction (eight sender accounts) and replies only when the block containing it is observed. (B4) *Keycloak*: a real Keycloak 26.4.0 issues each device an OpenID Connect access token whose claims carry its class, trust score and key hash, and the PEP calls the token-introspection endpoint [48] on every request; revocation disables the user. (B5) *OpenLDAP*: a real slapd directory holding the same attributes, queried once per request [49]. B4 and B5 also run with a PEP-side decision cache (“cached”, TTL 10 s), which is the usual way to cut their latency. (P) *BC-ZTA* as in Section 5.3. All systems use the same request format, signature check and policy, so differences come from where state is read and written.

5.5. Performance Metrics

(E1) End-to-end latency (mean, P95) from request send to decision, goodput (accepted requests per second), failures, and on-chain writes per second versus device count. (E2) Revocation enforcement delay: time from the revocation action until the PEP first denies the device. (E3) Security tests with 50 attempts each per run. (E4) Decision latency while the ledger is flooded with transactions. (E5) Decision latency, goodput and revocation enforcement while one of the four validators is stopped. Also gas per operation and, from the micro-benchmark, confirmation latency and ledger throughput.

5.6. Distributed Deployment

The end-to-end experiments E1–E5 ran on Google Cloud (us-central1-a) on seven VMs in one VPC without external addresses: four Besu validators (e2-medium, 2 vCPU, 4 GB, one validator each), a PEP/PDP host (e2-standard-2), the directory server for B1 (e2-small) and a controller (e2-standard-2) that runs the load generator and the experiment scripts and submits the on-chain transactions. Linux *tc netem* delays all outgoing traffic of each validator VM by 5 ms, to stand for a plant-local validator network, and the responses of the directory server by 40 ms, to stand for a remote central service; all other delay is the real VM-to-VM delay. Each of the 5 repetitions started a freshly generated chain and restarted the PEP for every device count, and the repetitions ran one after another on the same VMs. The validator-failure test (E5) stops one validator container for 30 s while 50 devices are served. A single cloud zone approximates independent hosts better than a shared CI runner, but it is not an industrial network: devices are emulated and co-tenancy in the cloud adds variance. Deployment scripts, configuration and raw results are in the public repository.

A second cloud run on the same VMs added Keycloak 26.4.0 (Docker, development mode, embedded H2 database, OIDC token introspection) and OpenLDAP slapd (Debian 12 package, one search per request) on the directory VM (B4, B5), the cached variants of both, a revocation test, a 25 s outage test, an open-loop PEP capacity test and the flood sweep of Section 6.9. Besu ran with a 2.5 GB heap for the flood sweep and all later runs; the first IAM repetition used the default 1 GB heap, which does not affect the PEP-side numbers reported here.

Table 3 lists the experiments with their conditions. All cloud runs used the first contract version, in which *AuditLog* and *grantSession* had no caller restriction (the hardening is described in Section 6.16), the unbound request signature and the per-event audit leaf; all used one resource with device-class and trust-score thresholds. Confidence intervals use the two-sided Student t value for n − 1 degrees of freedom (4.30 for n = 3, 2.78 for n = 5) on the run means, which assumes roughly normal run-to-run variation and is wide for n = 3.

TABLE 3. EXPERIMENT INVENTORY

Experiment	Runs	PEP VM	Besu heap	Notes
E1–E5 (6.1–6.7)	5	e2-standard-2	not recorded	first cloud run
IAM, revocation, outage (6.8)	3	e2-standard-2	1 GB, then 2.5 GB	Keycloak, OpenLDAP
Flood sweep (6.9)	2	e2-standard-2	2.5 GB	one attacker host
B6, quorum, fail-closed, halt, scale-out (6.10)	3	e2-standard-4	2.5 GB	second prototype run
Audit, registry (6.11–6.12)	100 trials; 5 passes	none (container)	n/a	local, no ledger

Runs are repetitions on a freshly started network. The Besu heap of the first run was not recorded.

6. Results and Discussion

6.1. Authentication

Table 4 and Fig. 3 show end-to-end latency (mean of 5 runs). At 50 devices BC-ZTA answers in 5.4 ms on average (P95 6.2 ms), versus 49.1 ms for the centralised PKI-IAM, 5.2 ms for the ledger-free ABAC store and 1382 ms for the synchronous blockchain. BC-ZTA is therefore 89% faster than the centralised baseline. This gap is almost entirely the injected 40 ms delay, i.e. the one wide-area round trip that BC-ZTA removes from the request path; it would shrink or grow with that delay, which we did not sweep, so the percentage is not a general speed-up. BC-ZTA also costs about 0.2 ms more than a local store (Merkle verification, hashing and audit bookkeeping), and is about 255× faster than checking every request on-chain. Even at 10 devices, where the ledger is not saturated, the synchronous design needs 1172 ms because each request waits for a block.

TABLE 4. END-TO-END LATENCY (MS, MEAN ± 95% CI OF 5 RUNS; P95)

System	Mean 50	P95 50	Mean 200	P95 200
PKI-IAM	49.1 ± 0.2	50.0	49.0 ± 0.2	50.9
ABAC	5.2 ± 0.3	6.0	4.1 ± 0.2	5.1
BC-only	1382 ± 17	2272	12094 ± 6401	20757
BC-ZTA	**5.4 ± 0.1**	**6.2**	**4.6 ± 0.2**	**5.8**

Columns give the number of devices; PKI-IAM responses are delayed by 40 ms (netem). With 5 runs the differences discussed in the text exceed the confidence intervals by an order of magnitude; no hypothesis tests were run.

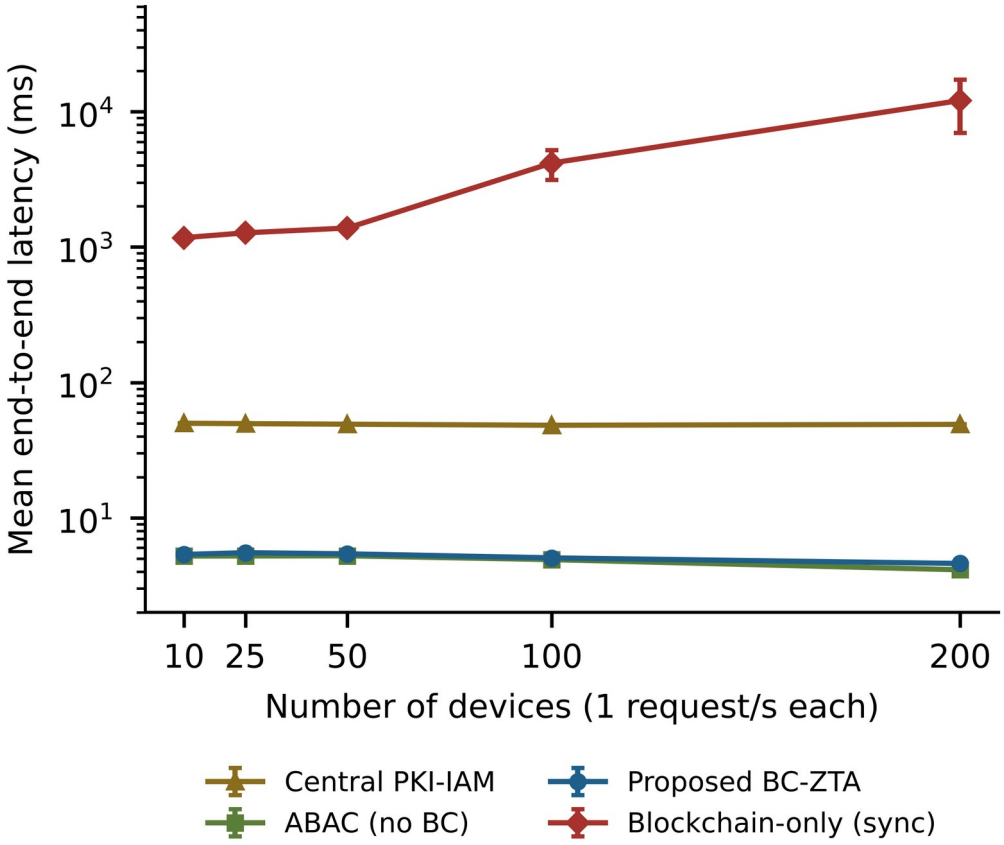


Fig. 3. Mean end-to-end latency versus number of devices (log scale; error bars: standard deviation of 5 runs; Table 4 gives 95% confidence intervals).

6.2. Authorization

The proposed design makes policy decisions from the edge cache. The measured cache-hit ratio was 96% (one miss per device per 30 s TTL), so 50 devices caused only 50 chain reads for about 1,250 requests; each miss costs one batched *eth_call* of 5.4 ms in the micro-benchmark. The PEP used 0.29 CPU cores at 50 devices and 0.65 at 200, against 0.53 for the local store. Every decision is covered by the audit batch: in the 200-device runs about 5,000 decisions per run were anchored in roughly 25 transactions. On-chain evaluation costs 44,685 gas as an estimate, but because it is a read, no fee or block wait is incurred.

6.3. Blockchain Throughput

The Besu micro-benchmark gave the following. Write confirmation takes 1522 ms on average at 10 tx/s (P95 2439 ms), close to the 2 s block period; a transaction submitted just after a block, as in the phase-locked sequential tests S1/S5, waits about 1980 ms. Reads cost 5.4 ms (P95 10.9 ms). Committed throughput tracks the offered load up to 50 tx/s (46.5 tx/s) but saturates near 60 tx/s on this host: at 100 and 200 tx/s offered only 62.7 and 59.1 tx/s commit and 43% of transactions at 200 tx/s were not confirmed within 60 s. Batch anchoring at 1 tx/s confirmed all 60 anchors in 941 ms on average. Because all validators share one 2-vCPU host, 60 tx/s is a lower bound: on the seven-VM deployment the synchronous baseline committed up to 112 tx/s (Section 6.5), almost twice as much.

6.4. Resource Utilization

Measured gas per operation is listed in Table 5. Registering a device costs 55,682 gas, whereas committing an epoch root costs 75,469 gas independent of batch size. Logging one event on-chain costs 182,008 gas, while anchoring a batch costs 134,747 gas, i.e. about 135 gas per record for a 1,000-record batch (≈1351× cheaper); the saving is proportional to the batch size and is ≈68× for the 50 records per batch of the 50-device experiments and ≈270× for 200 devices. At 400 devices and 1 event/s the calculation gives 34.6 M events/day: 6636 MB/day of ledger growth with per-event logging (192 B/entry) versus 13.8 MB/day with batch anchoring. Device-side CPU and memory were not measured. An IOTA-based DID study reports about 175% CPU on a Raspberry Pi against about 22% on a conventional computer while generating a DID [24], which is why we limit devices to one signature and a hash proof.

TABLE 5. MEASURED GAS PER CONTRACT OPERATION

Operation	Gas
IdentityRegistry.registerDID	55,682
IdentityRegistry.commitEpoch	75,469
IdentityRegistry.revokeDID	49,550
AccessControl.setPolicy	49,731
AccessControl.grantSession	71,689
TrustReputation.updateScore (quorum)	124,772
TrustReputation.flagAnomaly	95,868
AuditLog.logEvent (per event)	182,008
AuditLog.anchorBatch (per batch)	134,747

Hardhat EVM (london), optimizer 200 runs; first-use storage costs included.

6.5. Scalability

Fig. 4 shows goodput and on-chain write load against device count. Central PKI-IAM, ABAC and BC-ZTA served every request up to 200 devices with no failures. The synchronous blockchain kept up to 50 devices (goodput 50 req/s, 1.4 s mean latency) and at 100 devices (100 req/s, 4.2 s), but at 200 devices only 40% of the requests were answered in time, goodput fell to 81 req/s and 4331 requests (summed over 5 runs) failed or timed out. Its on-chain write rate (100 tx/s at 100 and 112 tx/s at 200 devices) is the measured ledger ceiling of this deployment, whereas BC-ZTA writes about 1.0 tx/s regardless of the number of devices. The latency of BC-ZTA stays flat from 10 to 200 devices (5.4 to 4.6 ms mean, P95 at most 6.4 ms), and the PEP used 0.65 of its two vCPUs at 200 devices, so neither the ledger nor the PEP was the limit at this load; larger populations were not tested. For comparison, a sharded design reaches 2,500 TPS at 700 nodes [18]; our approach instead lowers the required write rate.

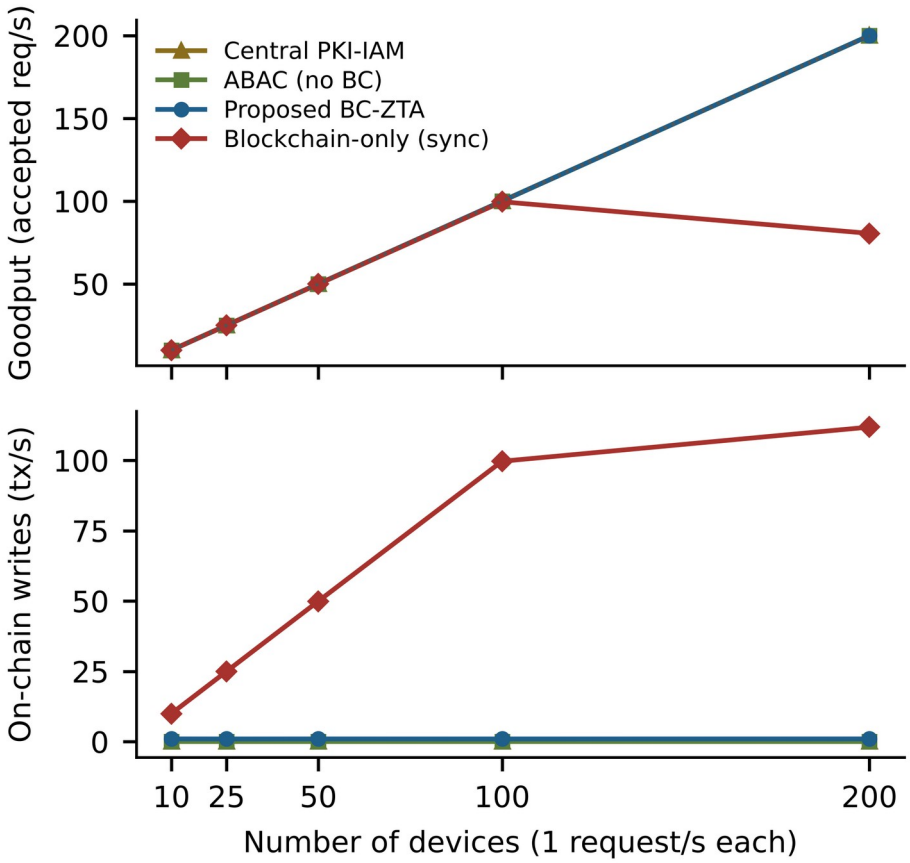


Fig. 4. Goodput (top) and on-chain writes per second (bottom) versus number of devices; dotted line: offered load.

6.6. Negative Security Tests

Table 6 lists the security tests run against the BC-ZTA prototype (5 runs, 50 attempts per test and run). All legitimate requests were accepted (no false rejects), and none of the forged, replayed, stale, unregistered, tampered, class-violating or low-trust requests was accepted. These are negative tests: each checks that one malformed request is refused by one mechanism. They do not model an adversary who compromises a gateway, holds the admin key, eclipses a node, controls several score voters or runs many fake devices. Those cases are covered by the argument in Section 4.6 and, for operator tampering with the audit trail, by the local experiment of Section 6.13; they were not tested on the cloud deployment. For a revoked device, 0 requests were accepted after the revocation transaction was confirmed, and the first denial came 15 ms after confirmation or sooner. Fig. 5 shows the revocation enforcement delay of each system: with event-driven invalidation BC-ZTA enforces a revocation after 1.25 s on average (worst case 2.23 s), about one block period and independent of the TTL, whereas TTL-only refresh with a 10 s TTL needs 6.6 s on average (worst case 11.5 s). The centralised and local baselines revoke in 57 ms and 22 ms because they hold the state in one place (the ABAC samples are bimodal, about 7 or 57 ms, which we did not trace); the blockchain-only value (1.35 s) includes the request that was in flight when the revocation was issued.

TABLE 6. NEGATIVE SECURITY TESTS ON THE BC-ZTA PROTOTYPE (5 RUNS)

Test	Attempts	Accepted
Legitimate requests (control)	250	250
Forged signature	250	0
Replayed request	250	0
Stale timestamp	250	0
Unregistered DID	250	0
Tampered Merkle proof	250	0
Class outside policy	250	0
Trust score below threshold	250	0

For the control row, Accepted is the desired outcome; for all other rows it must be 0.

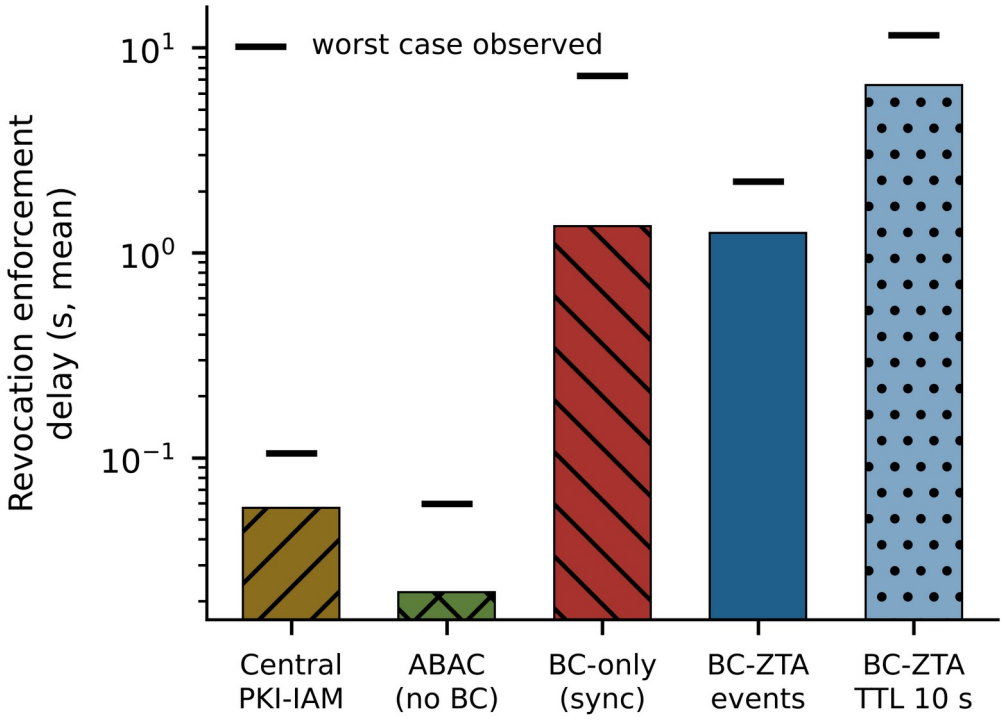


Fig. 5. Revocation enforcement delay (mean, log scale; marks: worst case observed).

6.7. Availability under Validator Failure

E5 stops one of the four validators for 30 s while 50 devices are served and one device is revoked during the outage. IBFT 2.0 with four validators tolerates $f = 1$ faulty validator and the PEP decides from its cache, so decisions should be unaffected. The test confirms this: all 3500 requests were answered and accepted (mean 5.4 ms, P95 6.8 ms, none late or failed), the revoked device was denied 1.76 s after the revocation was submitted, and the PEP kept anchoring audit batches (73 of 75 anchor transactions were confirmed when the statistics were read). Only one failure of one validator for 30 s was tested; network partitions, Byzantine validators and the loss of two validators were not.

6.8. Comparison with Production IAM Services

To replace the simple directory of B1 with real products, we added Keycloak 26.4 as an OpenID Connect token-introspection service [48] (B4) and OpenLDAP slapd as a directory queried once per request [49] (B5), each on the directory VM behind the same 40 ms netem delay, plus a variant of each with a 10 s decision cache at the PEP ("cached"). All five systems share the PEP, signatures and policy. Table 7 gives the results (3 repetitions). At 25 and 100 devices the uncached products need 48.3 ms (Keycloak) and 46.8 ms (OpenLDAP) per decision because every request crosses the emulated WAN, against 6.3 ms for BC-ZTA, which is 8× and 7× faster. Most of that gap is the removed WAN round trip: with a PEP-side cache the products reach 9.8–9.5 ms mean latency, and only their P95 (48 ms, one miss per device per TTL) stays WAN-bound. What matters is therefore the price of the cache rather than the mean latency. The cached products enforce a revocation only once the cache expires (mean 5.5 s, worst case 9.5 s for the 10 s TTL), whereas BC-ZTA's event-driven invalidation did so in 1.2 s in this run and 1.3 s in the five-run test of Section 6.6 (worst case 2.2 s), at the same low latency. The uncached products revoke faster, in 84 ms (Keycloak) and 72 ms (OpenLDAP), because the central service is consulted on every request. That is what BC-ZTA pays for edge autonomy: a plant that needs sub-second revocation must use the uncached products or a shorter block period.

TABLE 7. PRODUCTION IAM BASELINES: LATENCY (MEAN / P95, MS) AND REVOCATION DELAY

System	25 dev.	100 dev.	200 dev.	Revoc. (s)
Keycloak	49.7 / 52	48.3 / 50	123.4 / 304	0.08
Keycloak, cached	10.2 / 49	9.8 / 48	12.4 / 56	5.55
OpenLDAP	47.3 / 49	46.8 / 49	77.2 / 243	0.07
OpenLDAP, cached	10.1 / 47	9.5 / 47	10.6 / 50	5.20
BC-ZTA	**6.4 / 7**	**6.3 / 8**	117.2 / 255	**1.18**

Mean of 3 repetitions; 40 ms netem delay on the IAM services; revocation: 8 devices per run, mean of 24; cache TTL 10 s.

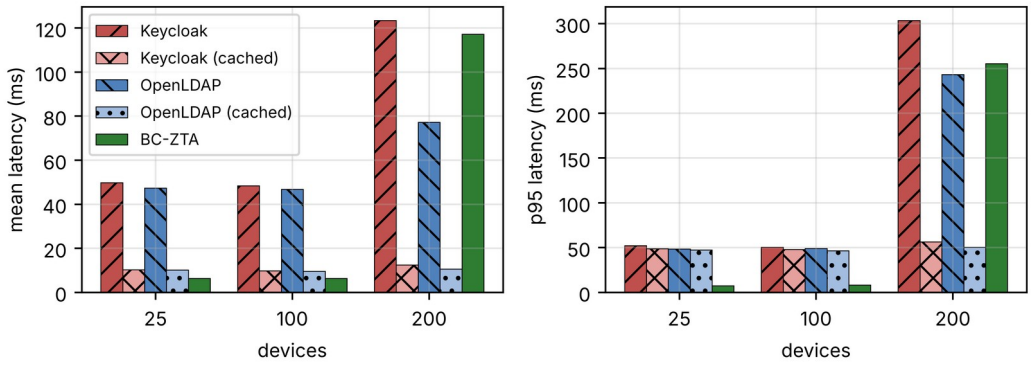


Fig. 6. Mean (left) and P95 (right) latency of Keycloak, OpenLDAP, their cached variants and BC-ZTA.

At 200 devices (one request per second each) the PEP, a single Node.js process on a 2-vCPU VM, nears its own capacity: BC-ZTA (117 ms mean, 255 ms P95) and uncached Keycloak (123 ms) degrade alike,

while the cached products do not. In the earlier run of Section 6.1 the same BC-ZTA configuration still decided in 4.6 ms at 200 devices, so this point sits at the saturation edge and varies from run to run. We read it as a limit of our single-process PEP and not of the architecture, since PEPs are stateless and can be replicated per gateway. An open-loop capacity test (200–1,600 req/s, 3 repetitions) shows goodput saturating at about 246–291 req/s for all three systems, with P95 latency of several seconds beyond it, so per-PEP capacity is set by signature verification and not by the IAM backend.

Table 8 reports the outage test: the central service is stopped 20 s into a 100 s run with 100 devices and restarted at 45 s. Without Keycloak or OpenLDAP no new decisions are possible: 0% of requests were granted during the outage with both uncached products and 4% with the cached variants, whose entries expired within 2 s of the stop (they kept serving only until the 10 s TTL ran out, and since requests are spread over the TTL, most entries had already expired). Service resumed 1 s after restarting OpenLDAP and 14 s after restarting Keycloak (JVM start-up). The PEP of BC-ZTA contacts no central service and granted 100% throughout (Fig. 7). This test removes a service that BC-ZTA does not use, and a design that caches decisions and receives pushed revocations from the same directory would also survive a short outage, so the fairer comparison is the loss of each design's own state source (Section 6.10). We did not test replicated or clustered Keycloak or OpenLDAP, which would reduce the effect at the cost of operating them. The test shows the single-point-of-failure property that the architecture removes, not that central IAM cannot be made highly available.

TABLE 8. CENTRAL-SERVICE OUTAGE (25 s, 100 DEVICES, 3 RUNS)

System	Granted during outage (%)	Recovery after restart (s)
Keycloak	0	14
Keycloak, cached	4	14
OpenLDAP	0	1
OpenLDAP, cached	4	1
BC-ZTA	**100**	**0**

Service stopped at 20 s and restarted at 45 s of a 100 s run; BC-ZTA has no central service (Keycloak was stopped on the directory VM, but BC-ZTA does not use it).

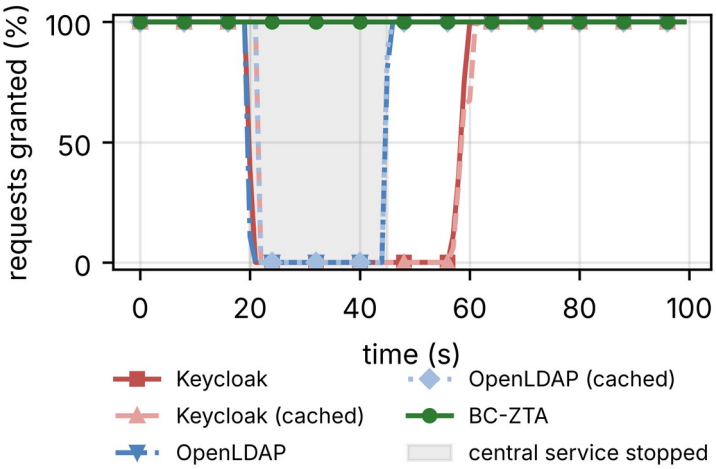


Fig. 7. Fraction of requests granted over time while the central IAM service is stopped (shaded) and restarted.

6.9. Ledger Flood Sweep

To test ledger flooding we swept the offered rate of audit-log transactions from 0 to 1,200 tx/s (16 spam wallets, pre-signed transactions sent in nonce order, 20 s per rate after a 5 s warm-up, 50 devices at 1 request/s, 2 repetitions), once with BC-ZTA and once with the synchronous design, which writes every decision as a transaction on the same ledger (Fig. 8). The committed rate saturates at about 193 tx/s for BC-ZTA and 215 tx/s for the synchronous design (peak at 200–400 tx/s offered) and then declines, to 106 and 152 tx/s at 1,200 offered, because Besu rejects most of the excess (“nonce too distant” and, at the highest rate, “transaction pool not enabled”; 50266 of 64802 submissions in the BC-ZTA run). The access latency of BC-ZTA does not follow: its mean stays at 5.8–6.3 ms and its P95 at no more than 7.9 ms with every request answered, since decisions never wait for the ledger. The mean latency of the synchronous design climbs from 1.6 s with an idle ledger to 24 s at 1,200 tx/s (P95 30 s), and 7% of its requests fail at the highest rate. A flood thus denies service to ledger-bound designs but only delays BC-ZTA’s own audit anchoring and revocation visibility, which we did not measure under flood.

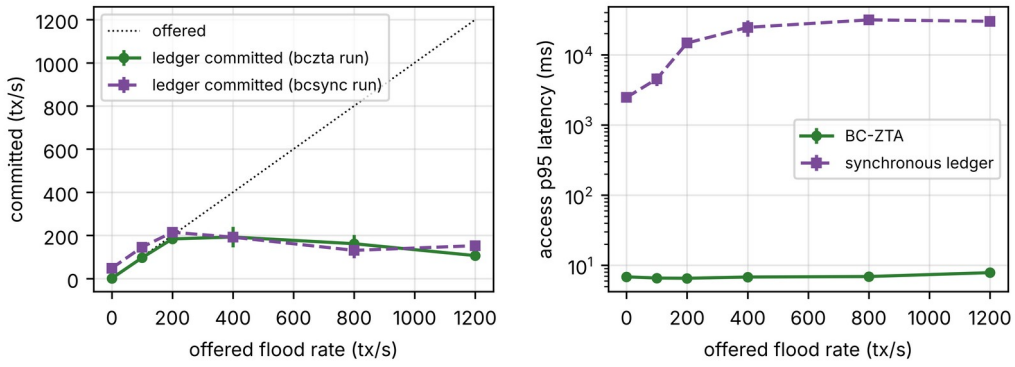


Fig. 8. Committed ledger rate (left) and access P95 latency (right) versus offered flood rate.

The flood generator targets the validator that the PEP also reads from. The Besu heap was raised to 2.5 GB after a first attempt with the default 1 GB heap ran out of memory in the synchronous run at 400 tx/s. A flood against all validators, an attacker with more accounts and a longer flood were not tested, and the committed rate of 100–215 tx/s is specific to our setup of four validators, a 2 s block period and two vCPUs per validator.

6.10. Robustness: Push Revocation, Read Quorum, Fail-Closed Bound, Consensus Halt and Scaling

The comparison with Keycloak and OpenLDAP leaves open whether the ledger is needed for what BC-ZTA does well. We therefore added baseline B6, a central directory (40 ms WAN delay) whose answers the PEP caches for 30 s and which pushes each revocation to the PEP over a server-sent-event stream, i.e. event-driven invalidation without a blockchain, and BC-ZTA with a read quorum of $f + 1 = 2$ validators (Section 3.8). The experiments ran 3 times, each on a fresh network, with the PEP host enlarged to e2-standard-4 (4 vCPU); Table 9 gives the results. B6 decides in 3.9 ± 0.4 ms at 100 devices against 4.2 ± 0.4 ms for BC-ZTA, so the two are equivalent for access latency. B6 enforces a revocation after 109 ms on average (worst case 154 ms), which is 11× faster than BC-ZTA with quorum (1.2 s, worst case 2.1 s), because BC-ZTA must wait for a block. The read quorum changes the mean latency at 100 devices from 4.2 ± 0.4 to 4.6 ± 1.1 ms, since only cache misses read from two nodes, and no quorum mismatch occurred in the runs. The result is that a trusted central operator who pushes revocations matches BC-ZTA's latency and revokes faster; what the ledger provides that B6 cannot is that no single operator holds identity, policy and audit records, and that anchored audit can be verified by the other consortium members. Where one operator is acceptable, B6 is the simpler design.

TABLE 9. CENTRAL DIRECTORY WITH PUSH REVOCATION (B6) AND READ QUORUM (3 RUNS)

System	25 dev.	100 dev.	200 dev.	Revoc. (ms)
B6 push	4.7 ± 0.3	3.9 ± 0.4	3.6 ± 0.3	109
BC-ZTA	4.5 ± 0.5	4.2 ± 0.4	4.0 ± 0.3	1252
BC-ZTA, quorum 2	4.6 ± 0.2	4.6 ± 1.1	4.0 ± 0.5	1162

Mean latency (ms) ± 95% CI of 3 runs; revocation: mean of 24 (B6) and 24 (quorum) samples; the BC-ZTA value is from the five-run test of Section 6.6. TTL 30 s (latency), 10 s (revocation).

Section 6.8 stopped Keycloak or OpenLDAP, which BC-ZTA does not use. The fair test is the loss of each design's own state source under the same cache policy (TTL 10 s, serve-stale bound $2 \cdot \text{TTL} = 20$ s, 50 devices): the PEP of BC-ZTA is cut off from its ledger node with a firewall rule (*iptables* DROP) for 50 s, and the directory of B6 is stopped for the same time (Table 10, Fig. 9, 3 runs). The two behave alike. Requests are granted from cache and then from stale state (89% and 100% granted in the first 10 s after the loss), the granted fraction falls to 0% and 0% once the last read is older than 20 s, and service resumes 1 s and 1 s after the source returns. The first denials come 9 s (BC-ZTA) and 12 s (B6) after the loss, not after 20 s, because the bound of each entry runs from its own last successful read, up to one TTL before the loss. The first reads that hit the unreachable ledger also wait for the 2.5 s read timeout, which is why BC-ZTA grants slightly less than B6 in the first window. The ledger therefore gives no extra availability in this test, since any cache with a stale bound behaves the same way in a short outage. The difference is that BC-ZTA state is replicated on four validators and the PEP could read from another node (we did not implement automatic failover), whereas the B6 directory is a single service unless it is replicated too.

TABLE 10. LOSS OF THE STATE SOURCE (TTL 10 S, STALE BOUND 20 S, 3 RUNS)

System (source lost)	Before	≤ TTL	TTL–2TTL	> 2TTL	Restored
BC-ZTA (ledger node)	100	89	15	0	100
B6 (directory)	100	100	20	0	100

Requests granted (%) in the windows after the loss of the state source; windows are relative to the time of the loss.

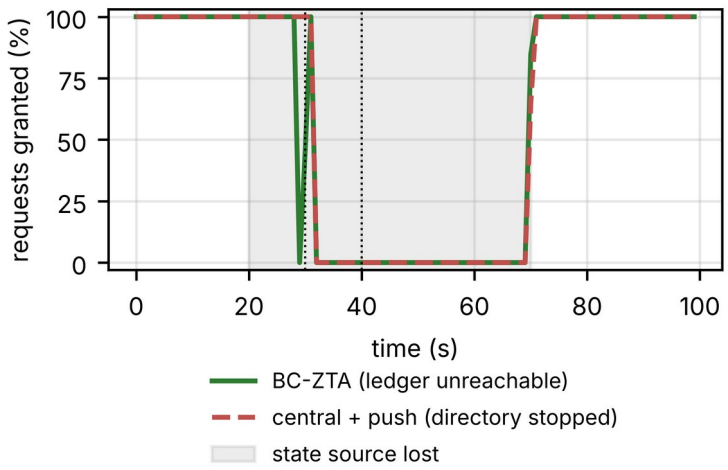


Fig. 9. Requests granted while the state source is lost (shaded); dotted lines: TTL and 2·TTL after the loss.

To test the consensus assumption we stopped two of the four validators for 40 s (IBFT 2.0 needs three, so block production halts) while 50 devices were served, and revoked a device 5 s after the stop. The PEP kept answering: 100% of requests were granted during the halt, because the cache and the reachable node serve the last committed state. The revoked device, however, stayed accepted until consensus resumed and the transaction was mined. Enforcement came 73.4 s after the validators were restarted and 108.4 s after submission (3 runs). The staleness bound of BC-ZTA therefore runs from finalisation of the revocation and not from its submission, so a consensus halt is a window in which revocations cannot take effect; a central directory is not affected by this particular failure. The delay is far longer than the halt itself (40 s): block production stayed frozen for 73 s after the restart. We did not test the cause. It is consistent with the IBFT 2.0 round-change rule, under which the timeout of a round doubles after each failed round (base 4 s in our configuration), so the validators needed one long round to agree again. Shorter base timeouts or supervised automatic restarts would reduce the delay, and an operator who needs bounded revocation latency during a halt must also enforce revocations at the PEP with a short-lived local deny list.

Finally, per-PEP capacity is not a limit of the architecture if gateways are replicated. We ran k = 1, 2, 4 PEP processes on one 4-vCPU VM, with devices sharded across them, and raised the offered rate (Fig. 10, Table 11). The highest offered rate served without any failed request and with a P95 below 50 ms was 200 req/s for k = 1, 400 req/s for k = 2, 400 req/s for k = 4. At 800 req/s offered, k = 1 is overloaded (goodput 457 req/s, P95 8.5 s) and k = 2 is overloaded (goodput 706 req/s, P95 7.2 s), whereas k = 4 delivers 770 req/s (96% of the offered rate) at a P95 of 436 ms while using 2.4 cores of the 4-vCPU VM. Beyond that the goodput of every configuration falls as the offered rate rises (overload collapse: requests queue behind signature verification and time out), so a deployment must keep the rate per gateway below its knee, which an admission limit or load balancer would enforce. Scale-out was measured on one host; across hosts the shared state is the ledger, which is read once per device per TTL.

TABLE 11. PEP SCALE-OUT (BC-ZTA, 3 RUNS)

PEPs	Clean rate (req/s)	Goodput at 800 (req/s)	P95 at 800 (ms)	CPU cores at 800
1	200	457	8464	0.7
2	400	706	7172	1.2
4	400	770	436	2.4

Clean rate = highest offered rate with no failed request and P95 < 50 ms; one VM; load generator on a separate VM; mean of the runs.

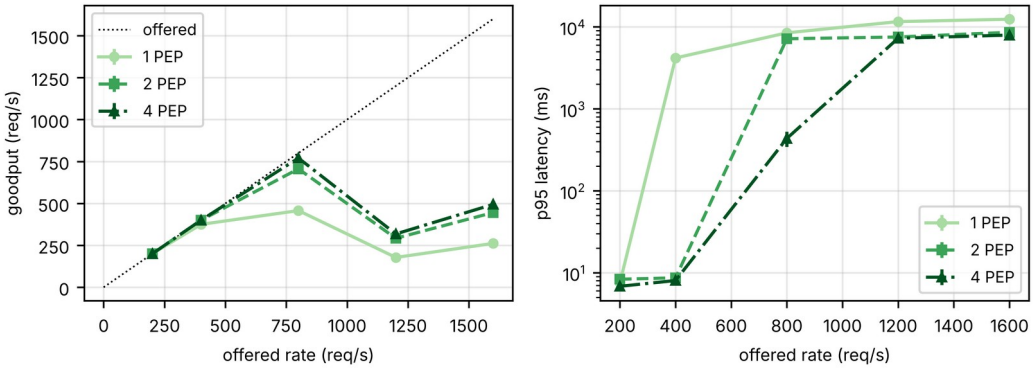


Fig. 10. Goodput (left) and P95 latency (right) versus offered request rate for 1, 2 and 4 PEP processes.

6.11. Independent Verification of the Audit Trail

A ledger-anchored audit log is only useful if someone outside the operator can check it. We tested this locally, without the cloud, with the same sorted-pair Merkle construction and keccak hashing as *AuditLog.anchorBatch*. For batches of 50 to 5,000 synthetic audit records, we computed the root and treated it as the value anchored on the ledger. An adversary who controls the stored log then changed one record, deleted one, inserted one, or swapped two adjacent ones (100 trials per case and size). The verifier recomputes the root from the log it is given and compares it with the anchored root. Table 12 shows that modification, deletion and insertion were caught in every trial at every batch size. Recomputing the whole batch took 1.4 ms for 50 records and 155 ms for 5,000 on one core, and a verifier interested in one record needs a proof of 192–416 bytes and 0.11–0.25 ms. A forged record presented with an honest proof was never accepted (0 of 100).

Two limits matter. First, swapping two adjacent records was caught in only 42–61% of the trials. Sorted-pair hashing makes the root of two sibling leaves independent of their order, so the root commits to the set of records and not to their order. Order can be checked only if every record carries a sequence number that the verifier inspects; the measured prototype's records carry a timestamp and a nonce, not a counter (the v2 format below adds one). Second, the anchor covers a record only from the moment its batch is closed and anchored. A record falsified before that, a batch whose root was never anchored, and a log the operator refuses to hand over are outside what the ledger proves: it shows that a root existed at a given block, not that the log is complete. A central database log without an external commitment gives a third party nothing to compare against, so a rewritten row is undetectable by construction (we did not measure this case). This test shows that the tamper-evidence claim holds for the mechanism; it does not compare BC-ZTA with a transparency log or with an operator-published Merkle log of the kind in [39], which offer similar evidence without consensus. What the ledger adds over these is a publication medium that no single party controls.

We then tested the remedy for the ordering gap. In record format v2 (*AUDIT_V2*, default off in the PEP) each record also carries a gateway identifier, a per-gateway sequence number, the decision and a hash of the device signature, and the verifier additionally requires the sequence numbers of a presented batch to run 0 to n − 1 in order. In the same local setting (100 trials per case and size, batches of 50 to 5000), modification, deletion, insertion, adjacent swaps and swaps followed by renumbering were detected in 2500 of 2500 trials, with no false alarm on the untouched batch. The hash of the device signature makes a record attributable to a signed request, so a gateway cannot later claim it never saw one; it does not prove that the gateway recorded every request it received, and a gateway authorised to anchor can still omit records. These formats were tested locally and were not part of the cloud runs.

TABLE 12. DETECTION OF TAMPERING WITH ANCHORED AUDIT BATCHES (100 TRIALS PER CELL)

Batch	Modify	Delete	Insert	Swap	Full (ms)	Proof (B)	Proof (ms)
50	100/100	100/100	100/100	50/100	1.4	192	0.11
100	100/100	100/100	100/100	61/100	2.6	224	0.13
500	100/100	100/100	100/100	48/100	14.1	288	0.17
1000	100/100	100/100	100/100	52/100	29.3	320	0.19
5000	100/100	100/100	100/100	42/100	155.1	416	0.25

Detected/trials. Run on a 2-vCPU container with Node v22.23.4, not on the cloud VMs; the log and its proofs are held by the adversary, the root by the ledger.

6.12. Registry Size

The end-to-end runs used up to 200 devices. To see whether proof handling would limit larger sites, we measured the Merkle epoch tree of the device registry from 1,000 to 1,000,000 entries in the same local test (Table 13). The proof grows from 320 to 640 bytes (depth 10 to 20) and one proof takes 0.18 to 0.43 ms to verify on one core, which is 2,307–5,474 verifications per second per core with a JavaScript hash. A PEP that verifies a proof on every cache miss therefore pays well under a millisecond even at one million devices, and the cost grows with the logarithm of the registry. The costs that grow faster are building the tree, which the registrar does once per epoch off the request path (27 s for 1,000,000 entries on one core), and the cache itself. These numbers cover proof handling only. They contain no network, no ledger and no contention between gateways, so they do not replace an end-to-end run, and our measured claim for the whole system stays at 200 devices.

TABLE 13. MERKLE EPOCH TREE VERSUS REGISTRY SIZE (ONE CORE, LOCAL TEST)

Devices	Depth	Proof (B)	Verify (ms)	Build (s)
1,000	10	320	0.18	0.02
10,000	14	448	0.26	0.22
100,000	17	544	0.33	2.25
1,000,000	20	640	0.43	27.46

Random 32-byte leaves, 2,000 random proofs per size, mean of 5 passes.

6.13. Operator Tampering, Static Analysis and Admin Control

Section 6.11 tested whether a verifier can check a batch it is handed. The harder question is what a malicious operator can change afterwards and who would notice. We compared three ways of keeping the same 1,000 audit records in 10 batches: a mutable central log with nothing published outside the operator (the audit trail of baseline B6), a hash-chained log whose head the operator publishes on its own site, and BC-ZTA, with batch roots anchored in the real *AuditLog* contract on a local Hardhat network. The operator can edit the database and holds the contract admin key but is not part of the validator quorum. The verifier sees only what each design publishes outside the operator's control. Table 14 lists whether the verifier notices.

TABLE 14. OPERATOR TAMPERING: WHAT AN OUTSIDE VERIFIER NOTICES (LOCAL TEST)

Operator action	Central log	Hash chain	BC-ZTA anchors
Edit one record	Not detected	Detected	Detected
Delete one record	Not detected	Detected	Detected
Insert a forged record	Not detected	Detected	Detected
Rewrite the log and recompute the chain	Not detected	Not detected	Detected
Anchor a replacement root as admin	Not detected	Detected	Detected
Gateway drops a record before anchoring	Not detected	Not detected	Not detected

Logic-level comparison on the real AuditLog contract; the verifier rules are ours. It states what each design publishes, not how fast it runs.

The central log shows nothing, because there is nothing outside the operator to compare it with. The hash chain catches edits, deletions and insertions but not a full rewrite followed by a new head. The anchored design catches all five edits, including the case where the operator authorises itself and anchors a replacement root: the original root stays on the ledger, and the number of anchors no longer

matches the number of batches. No design here notices a gateway that drops a record before it is anchored, as Section 4.6 says. The comparison is fair only up to a point. A central log with an independent monitor that keeps earlier heads could detect the same edits; the ledger's advantage is that no such trusted party has to exist and that the roots are held by independent validators.

We also ran the static analyser Slither (version 0.11.6) on the contracts. It reported no high-impact findings, 1 medium finding (strict equality on a hash comparison in *verifyInBatch*, which is intended), 3 low findings (block-timestamp comparisons, which are how session lifetimes and the update rate limit work), 2 informational findings and 6 gas suggestions. A symbolic tool such as Mythril, formal verification and an independent manual audit were not done.

Finally, *AccessControl* and *AuditLog* can now hand the admin role to a multisig contract. In the unit tests a 2-of-3 *MultiSigAdmin* applies a policy change and a gateway authorisation only after two owners confirm, and the old single key is refused. This is covered by unit tests (18 pass) and was not part of the cloud measurements.

6.14. Comparison with Baselines

BC-ZTA is 0.2 ms slower than a ledger-free ABAC store at 50 devices, but adds decentralised identity, shared policy and tamper-evident anchored audit that B2 lacks. Against the centralised PKI-IAM it is 89% faster at 50 devices (a gap set by the injected 40 ms of netem delay on the directory, which was not varied) and removes the central directory, and against the synchronous blockchain it avoids about 1.4 s per decision and the ledger ceiling. Against real Keycloak and OpenLDAP (Section 6.8) it is 8× and 7× faster at 100 devices, but a PEP-side cache gives similar mean latency at the cost of stale revocations, and a central directory with pushed revocations (Section 6.10) matches its latency and revokes faster. The price is revocation latency: about 1.3 s with events (one block) instead of about 84 ms for a central store. The Ethereum-based framework mentioned in Section 1 pays about 25 ms extra per request even at small scale [17], in line with our finding that consensus on the request path is the bottleneck.

6.15. Discussion

The central trade-off is bounded revocation staleness in exchange for latency. The push baseline of Section 6.10 shows that the trade-off is not specific to the ledger: a trusted central operator gets the same latency and faster revocation, so the ledger has to be justified by who controls identity state and who can verify the audit, not by speed.

6.16. Limitations and Deployment Considerations

The main limitations are these. (i) The seven VMs sit in one zone of one cloud provider, so inter-VM delay is that of a data-centre network plus the injected netem delay, not that of a plant network or a WAN, and absolute latency and throughput are specific to the VM sizes. (ii) B1–B3 are simple re-implementations; Keycloak ran in development mode (single node, embedded H2 database) and OpenLDAP as a single server, both behind an emulated 40 ms WAN, so tuned or clustered deployments may behave differently, particularly for availability. (iii) Devices are emulated by a load generator, with up to 200 devices at 1 request/s end to end; larger registries were tested only as a proof-handling microbenchmark (Section 6.12), and no physical devices, industrial protocols, secure elements or bursty, churn-heavy workloads were used. (iv) One plant authority key issues registrations and revocations, so authority compromise is out of scope. (v) The availability tests are narrow: Besu rejected part of the E4 flood (50266 of 64802 submissions at 1,200 tx/s), E5 stops one validator once for 30 s and Section 6.10 stops two for 40 s, and there is no network partition or Byzantine validator. (vi)

Experiments have five repetitions (three for the IAM tests, two for the flood sweep) on the same VMs, the revocation tests use 8 devices per run, and a single-process PEP saturates at roughly 250–290 req/s, so larger sites need several PEPs (scale-out was measured on one host only). (vii) Trust scores are voted values, not learned from data, and micro-segmentation (R5) is not implemented, so only R1, R2 and R6 are evaluated end to end. (viii) At 200 devices the single-process PEP is at its capacity: BC-ZTA averaged 4.6 ms in one run and 117 ms (P95 255 ms) in another, so results at that point depend on run conditions; the headline latencies are those at 50 and 100 devices. Several effects were seen but not explained: the cause of the 73 s that revocation needed after consensus restart, and the bimodal ABAC revocation samples. (ix) No real industrial protocol (OPC UA, Modbus, MQTT) was used and no plant traffic was replayed, so the request format is ours, and 3 to 5 repetitions per point give wide intervals. Comparisons with published numbers are indicative only, since workloads and testbeds differ.

The security analysis (Section 4.6) adds further limits. The nonce cache is per gateway and, in the measured configuration, the signed request carries no gateway identifier, so a captured request can be replayed once at another gateway within Δ. The released code has an optional gateway binding (*GATEWAY_BIND*, default off) in which the device signs the target gateway; a message-level check confirms that a request bound to one gateway is rejected at another, but the end-to-end cloud runs used the unbound configuration. The default read path trusts one ledger node (the quorum option is implemented, and its cost is measured in Section 6.10, but it is not part of the default measurements). Audit completeness is not protected against a compromised gateway, and the Merkle epoch proof shows enrolment, not freshness. The device-side cost of signing and storing a proof was not measured, so compatibility with constrained devices (D5) is an argument, not a result. Each request is signed individually, which suits access decisions but not high-rate telemetry, where a session credential issued after one verified request would be used instead. The security analysis of Section 4.6 is an argument under stated assumptions and not a machine-checked proof; a protocol analysis with a tool such as Tamarin or ProVerif would be the natural next step. All statistical statements rest on 2–5 repetitions with standard deviations or 95% confidence intervals and no hypothesis tests. These limits are common: a recent review of 33 trust-score studies for zero-trust access control found that about two thirds rely on simulation only, fewer than a fifth consider adversarial threats, and only a few test beyond 1,000 nodes [50].

Emulation limits what the numbers say about absolute latency, but less about the comparison. All designs ran on the same testbed, and the quantities that separate them (a consensus round on the request path, a wide-area round trip to a central directory, and the time a revocation needs to reach the enforcement point) are set by the block interval and the delay of the path, not by device hardware. A plant WAN with larger or more variable delay would lengthen the centralised designs and the cache-miss path of BC-ZTA but not its cached path, and a multi-site Besu network would lengthen block finality and therefore revocation. We state this as an argument; only a multi-site measurement could confirm it.

Several points matter for deployment. Validators must be run by independent parties (plant, integrator, vendor) for the Byzantine assumption to mean anything. Device keys should sit in a secure element or TPM, since the identity guarantees are only as strong as key custody; rotation goes through *updateKey* by the enrolment authority, and neither the cost of rotating a fleet nor the custody of the authority key was evaluated. The replay window assumes that gateway and device clocks differ by less than half of Δ, which constrained devices can meet only with a time source such as NTP from the gateway; we did not

evaluate this. On-chain records hold key hashes, scores and event hashes, not process data, which keeps operational data off the shared ledger. Even so, a DID that appears in a registry entry or a score update is a stable pseudonym, so any member with ledger access can link on-chain events to one device, and resource identifiers in a policy show which assets exist; validators and other consortium members can read all of this. Deployments that treat the device inventory as sensitive should use per-organisation private data or rotate pseudonymous identifiers per epoch. We evaluated none of these and did not analyse legal privacy requirements. In the measured prototype *AuditLog* and *grantSession* were callable by any account. The released code now restricts both to an admin-managed set of gateway addresses (17 unit tests pass, including unauthorised-caller reverts); the cloud measurements predate this change, and re-measuring gas on the hardened contracts shows +1.2% (*logEvent*), +1.6% (*anchorBatch*) and +3.1% (*grantSession*), so the gas figures and ratios in this paper are unaffected in substance. A gateway that is authorised can still write false records, so the allow-list limits who may write, not what is written. The prototype uses IBFT 2.0; Besu also offers QBFT, which we did not test. Consensus is off the request path, so we expect PEP-side latency to be unchanged, but revocation delay and halt behaviour depend on block period and view-change logic and would need re-measuring. Gas on a permissioned network has no price, so the gas figures are a relative measure of on-chain work per operation (the ratios between designs are the useful result), and they would translate to cost only on a public chain. Finally, the fail-closed rule of Section 3.7 trades availability for safety during ledger partitions and should be tuned against the plant's uptime requirements.

7. Conclusion

We presented BC-ZTA, a blockchain-assisted zero-trust architecture that keeps consensus off the request path by using edge-cached, Merkle-verified state and asynchronous batched anchoring. Four Solidity contracts pass 17 unit tests, and measured gas shows that batch anchoring is 68–1351× cheaper per record than per-event logging for batches of 50–1,000 records. On seven Google Cloud VMs with a four-validator Besu network, BC-ZTA decided in 5.4 ms on average at 50 devices, served 200 devices without failed requests with about one on-chain write per second, and kept deciding while one validator was stopped; at 200 devices, however, the single-process enforcement point is at capacity and its latency varied between runs (4.6 and 117 ms mean), so we regard 50–100 devices per enforcement point as the reliable operating range and rely on replication beyond that. The synchronous design needed 1.4 s at 50 devices and answered only 40% of requests at 200 devices. The price is a revocation delay of about 1.3 s with event-driven invalidation. We also added a central directory with pushed revocations to test whether the ledger is needed for speed. It has the same latency and revokes in 109 ms, so the case for the ledger rests on removing any single operator's control over identity state and on audit that third parties can verify, not on performance. A fail-closed bound of twice the cache lifetime, a read quorum of two validators, replicated gateways and the behaviour during a consensus halt (revocations wait for consensus, here 108 s on average) are evaluated in Section 6.10. All forged, replayed, stale, unregistered, tampered and policy-violating requests were rejected. In a local test, tampering with anchored audit batches (modification, deletion, insertion) was caught in every trial, whereas swapping adjacent records was not reliably caught because the root commits to the set of records and not their order; adding a sequence number to each record closed that gap in the local test. Requirements R4 (continuous trust assessment) and R5 (micro-segmentation) are design-level only and are not evaluated. The evaluation used cloud VMs in a single zone, injected network delay and an emulated device population. Future work includes a multi-site Besu deployment with real network delay, physical

devices and industrial protocols, clustered production IAM baselines, learned trust scoring and post-quantum signatures.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper. [Authors: please confirm.]

CRediT authorship contribution statement

Syed Irfan Raza Naqvi: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review & editing, Visualization. **Muhammad Umar Aftab:** Conceptualization, Supervision, Writing – review & editing.

Funding

[Funding source and grant number, or: This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.]

Data availability

The Solidity contracts, the PEP and load-generator code, the Google Cloud deployment scripts and all raw results used in this paper are available at https://github.com/irfanrazanaqvi/bc-zta-iiot-experiments (archived release with DOI: [to be added by the authors before submission]).

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work the authors used Claude (Anthropic) to assist with writing experiment code and automation scripts, data analysis, and language editing of the manuscript. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article. [Authors: please confirm this statement is accurate.]

Acknowledgements

[Acknowledgements to be added by the authors.]

References

[1] F. Federici, D. Martintoni, and V. Senni, “A zero-trust architecture for remote access in industrial IoT infrastructures,” Electronics, vol. 12, no. 3, Art. no. 566, 2023, doi: 10.3390/electronics12030566.

[2] C. Zanasi, S. Russo, and M. Colajanni, “Flexible zero trust architecture for the cybersecurity of industrial IoT infrastructures,” Ad Hoc Netw., vol. 156, Art. no. 103414, 2024, doi: 10.1016/j.adhoc.2024.103414.

[3] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, “Zero trust architecture,” Nat. Inst. Standards Technol., Gaithersburg, MD, USA, Spec. Publ. 800-207, Aug. 2020, doi: 10.6028/NIST.SP.800-207.

[4] M. L. Gambo and A. Almulhem, “Zero trust architecture: A systematic literature review,” J. Netw. Syst. Manage., vol. 34, Art. no. 25, 2026, doi: 10.1007/s10922-025-09998-x.

[5] S. Mushtaq, M. Mohsin, and M. M. Mushtaq, “A systematic literature review on the implementation and challenges of zero trust architecture across domains,” Sensors, vol. 25, no. 19, Art. no. 6118, 2025, doi: 10.3390/s25196118.

[6] M. A. Azad, S. Abdullah, J. Arshad, H. Lallie, and Y. H. Ahmed, “Verify and trust: A multidimensional survey of zero-trust security in the age of IoT,” Internet Things, vol. 27, Art. no. 101227, 2024, doi: 10.1016/j.iot.2024.101227.

[7] S. Mthethwa, M. T. Dlamini, and E. Jembere, “Zero trust for NHIs based on robust identity and access management for a resilient IoT future,” Sensors, vol. 26, no. 8, Art. no. 2392, 2026, doi: 10.3390/s26082392.

[8] M. Asim, N. Tariq, A. I. Awad, F. Waheed, S. Bukhari, A. Rafay, and H. H. Song, “Lightweight and privacy-aware decentralized identity management for Industrial Internet of Things applications,” Future Gener. Comput. Syst., vol. 186, Art. no. 108753, 2027 (online Aug. 2026), doi: 10.1016/j.future.2026.108753.

[9] E. B. Fernández and A. Brazhuk, “A critical analysis of zero trust architecture (ZTA),” Comput. Stand. Interfaces, vol. 89, Art. no. 103832, 2024, doi: 10.1016/j.csi.2024.103832.

[10] Y.-W. Ma and P.-H. Chiu, “A novel risk-based access control engine in zero trust architecture for IoT network,” Int. J. Inf. Secur., vol. 24, Art. no. 124, 2025, doi: 10.1007/s10207-025-01030-2.

[11] M. A. Aleisa, “Blockchain-enabled zero trust architecture for privacy-preserving cybersecurity in IoT environments,” IEEE Access, vol. 13, pp. 18660–18676, 2025, doi: 10.1109/ACCESS.2025.3529309.

[12] S. Nie, J. Ren, R. Wu, P. Han, Z. Han, and W. Wan, “Zero-trust access control mechanism based on blockchain and inner-product encryption in the Internet of Things in a 6G environment,” Sensors, vol. 25, no. 2, Art. no. 550, 2025, doi: 10.3390/s25020550.

[13] G. Ali, S. Shah, M. Elaffendi, and N. Ahmad, “Blockchain-based access management framework for interoperable digital twins in industrial IoT,” Front. Blockchain, vol. 8, Art. no. 1693926, 2025, doi: 10.3389/fbloc.2025.1693926.

[14] M. Essaid and H. Ju, “Blockchain solutions for enhancing security and privacy in Industrial IoT,” Appl. Sci., vol. 15, no. 12, Art. no. 6835, 2025, doi: 10.3390/app15126835.

[15] D. Li, H. Li, N. Crespi, R. Minerva, M. Li, W. Liang, and K. C. Li, “Hyper-IIoT: A smart contract-inspired access control scheme for resource-constrained Industrial Internet of Things,” IEEE Trans. Sustain. Comput., vol. 10, no. 5, pp. 820–829, 2025, doi: 10.1109/TSUSC.2025.3542466.

[16] Z. Guo, “Blockchain-enhanced smart contracts for formal verification of IoT access control mechanisms,” Alexandria Eng. J., vol. 118, pp. 315–324, 2025, doi: 10.1016/j.aej.2024.12.109.

[17] A. Singh, V. Pareek, and A. Sharma, “Blockchain-enabled zero trust framework for securing FinTech ecosystems against insider threats and cyber attacks,” arXiv:2507.19976, 2025.

[18] M. M. Salim, M. Kim, S. K. Singh, and J. H. Park, “Zero-trust blockchain-enabled framework for scalable and secure IoT networks,” Future Gener. Comput. Syst., vol. 175, Art. no. 108093, 2026, doi: 10.1016/j.future.2025.108093.

[19] A. Sharma, S. Rani, and W. Boulila, “Blockchain-based zero trust networks with federated transfer learning for IoT security in industry 5.0,” PLOS ONE, vol. 20, no. 6, Art. no. e0323241, 2025, doi: 10.1371/journal.pone.0323241.

[20] S. Mishra, T. S. M. Aldafas, and N. S. Alshammari, “A zero-trust digital twin framework for privacy-preserving multi-dataset intrusion detection in industrial IoT with lightweight blockchain auditing,” Sci. Rep., vol. 16, Art. no. 15236, 2026, doi: 10.1038/s41598-026-42041-w.

[21] Z. Zhai, J. Liu, X. Liu, Y. Mao, X. Zhang, J. Ma, and C. Jin, “A lightweight authentication method for Industrial Internet of Things based on blockchain and Chebyshev chaotic maps,” Future Internet, vol. 17, no. 8, Art. no. 338, 2025, doi: 10.3390/fi17080338.

[22] K. Ding, T. Xie, K. Gai, C. Guo, L. Lei, D. Wang, J. Yu, L. Zhu, and W. Meng, “Verifiable decentralized identity-based meta-computing in Industrial Internet of Things (IIoT),” J. Syst. Archit., vol. 162, Art. no. 103391, 2025, doi: 10.1016/j.sysarc.2025.103391.

[23] W. Xu, J. Deng, J. Yu, S. Mao, Y. Li, Z. Peng, and B. Xiao, “Blockchain-based verifiable decentralized identity for intelligent flexible manufacturing,” IEEE Internet Things J., vol. 12, no. 16, pp. 32366–32378, 2025, doi: 10.1109/JIOT.2025.3576735.

[24] T. Ramírez-Gordillo, A. Maciá-Lillo, F. A. Pujol, N. García-D’Urso, J. Azorín-López, and H. Mora, “Decentralized identity management for Internet of Things (IoT) devices using IOTA blockchain technology,” Future Internet, vol. 17, no. 1, Art. no. 49, 2025, doi: 10.3390/fi17010049.

[25] S. I. R. Naqvi, Z. J. Bin, A. Mohsin, M. P. Akhter, and M. M. ur Rahman, “DBoT: Enhancing scalability and privacy in intelligent building spaces through a direct acyclic graph blockchain,” in Proc. 17th Int. Conf. Open Source Syst. Technol. (ICOSST), 2023, doi: 10.1109/ICOSST60641.2023.10414234.

[26] R. Islam, R. Bose, S. Roy, A. A. Khan, S. Sutradhar, S. Das, F. Ali, and A. A. AlZubi, “Decentralized trust framework for smart cities: A blockchain-enabled cybersecurity and data integrity model,” Sci. Rep., vol. 15, Art. no. 23454, 2025, doi: 10.1038/s41598-025-06405-y.

[27] T. Wan, B. Shi, and H. Wang, “A continuous authentication scheme for zero-trust architecture in industrial internet of things,” Alexandria Eng. J., vol. 122, pp. 555–563, 2025, doi: 10.1016/j.aej.2025.03.012.

[28] U. Panahi, “ZT-IIoT: An integration-centric zero trust architecture for secure IT/OT convergence in industrial information systems,” Int. J. Crit. Infrastruct. Prot., vol. 55, Art. no. 100917, 2026, doi: 10.1016/j.ijcip.2026.100917.

[29] N. N. Hari, P. Krishnan, K. Jain, A. K. J. Saudagar, P. Prabu, and R. C. Poonia, “A three-tier microsegmentation framework for enterprise networks under Zero Trust Architecture,” Alexandria Eng. J., vol. 141, pp. 150–164, 2026, doi: 10.1016/j.aej.2026.03.014.

[30] N. F. Syed, S. W. Shah, A. Shaghaghi, A. Anwar, Z. Baig, and R. Doss, “Zero trust architecture (ZTA): A comprehensive survey,” IEEE Access, vol. 10, pp. 57143–57179, 2022, doi: 10.1109/ACCESS.2022.3174679.

[31] K. Li et al., “Zero-trust foundation models: A new paradigm for secure and collaborative artificial intelligence for Internet of Things,” arXiv:2505.23792, 2025.

[32] S. Ghasemshirazi, G. Shirvani, and M. A. Alipour, “Zero trust: Applications, challenges, and opportunities,” arXiv:2309.03582, 2023.

[33] A. K. Peepliwal et al., “A prototype model of zero-trust architecture blockchain with EigenTrust-based practical Byzantine fault tolerance protocol to manage decentralized clinical trials,” arXiv:2408.16885, 2024.

[34] A. Kulkarni, N. A. Hazari, and M. Y. Niamat, “A zero trust-based framework employing blockchain technology and ring oscillator physical unclonable functions for security of field programmable gate array supply chain,” IEEE Access, vol. 12, pp. 89322–89338, 2024, doi: 10.1109/ACCESS.2024.3418572.

[35] D. Cooper, S. Santesson, S. Farrell, S. Boeyen, R. Housley, and W. Polk, “Internet X.509 public key infrastructure certificate and certificate revocation list (CRL) profile,” Internet Eng. Task Force, RFC 5280, May 2008. [Online]. Available: https://www.rfc-editor.org/rfc/rfc5280

[36] S. Santesson, M. Myers, R. Ankney, A. Malpani, S. Galperin, and C. Adams, “X.509 Internet public key infrastructure online certificate status protocol—OCSP,” Internet Eng. Task Force, RFC 6960, Jun. 2013. [Online]. Available: https://www.rfc-editor.org/rfc/rfc6960

[37] D. Eastlake 3rd, “Transport layer security (TLS) extensions: Extension definitions,” Internet Eng. Task Force, RFC 6066, Jan. 2011. [Online]. Available: https://www.rfc-editor.org/rfc/rfc6066

[38] B. Laurie, A. Langley, and E. Kasper, “Certificate transparency,” Internet Eng. Task Force, RFC 6962, Jun. 2013. [Online]. Available: https://www.rfc-editor.org/rfc/rfc6962

[39] M. A. Yağız, F. Horasan, and A. H. Yurttakal, “Lightweight tamper-evident log integrity verification for IoT edge environments: A Merkle-tree pipeline with adaptive chunking,” arXiv:2605.00065, 2026.

[40] M. Sporny, D. Longley, M. Sabadello, D. Reed, O. Steele, and C. Allen, “Decentralized identifiers (DIDs) v1.0,” W3C Recommendation, Jul. 2022. [Online]. Available: https://www.w3.org/TR/did-core/ [Online]. Available: https://www.w3.org/TR/did-core/

[41] A. Ouaddah, A. Abou Elkalam, and A. Ait Ouahman, “FairAccess: A new blockchain-based access control framework for the Internet of Things,” Secur. Commun. Netw., vol. 9, no. 18, pp. 5943–5964, 2016, doi: 10.1002/sec.1748.

[42] O. Novo, “Blockchain meets IoT: An architecture for scalable access management in IoT,” IEEE Internet Things J., vol. 5, no. 2, Apr. 2018, doi: 10.1109/JIOT.2018.2812239.

[43] E. Androulaki et al., “Hyperledger Fabric: A distributed operating system for permissioned blockchains,” in Proc. 13th EuroSys Conf., Porto, Portugal, Apr. 2018, Art. no. 30, doi: 10.1145/3190508.3190538.

[44] H. Moniz, “The Istanbul BFT consensus algorithm,” arXiv:2002.03613, 2020.

[45] M. Castro and B. Liskov, “Practical Byzantine fault tolerance,” in Proc. 3rd USENIX Symp. Oper. Syst. Design Implement. (OSDI), New Orleans, LA, USA, Feb. 1999, pp. 173–186. [Online]. Available: https://www.usenix.org/conference/osdi-99/practical-byzantine-fault-tolerance

[46] M. Dahlmanns, J. Lohmöller, I. B. Fink, J. Pennekamp, K. Wehrle, and M. Henze, “Easing the conscience with OPC UA: An Internet-wide study on insecure deployments,” in Proc. ACM Internet Meas. Conf. (IMC), Virtual Event, Oct. 2020, pp. 101–110, doi: 10.1145/3419394.3423666.

[47] B. Blanchet, “Modeling and verifying security protocols with the applied pi calculus and ProVerif,” Found. Trends Privacy Secur., vol. 1, no. 1–2, pp. 1–135, 2016, doi: 10.1561/3300000004.

[48] J. Richer, Ed., “OAuth 2.0 token introspection,” Internet Eng. Task Force, RFC 7662, Oct. 2015. [Online]. Available: https://www.rfc-editor.org/rfc/rfc7662

[49] J. Sermersheim, Ed., “Lightweight Directory Access Protocol (LDAP): The protocol,” Internet Eng. Task Force, RFC 4511, Jun. 2006. [Online]. Available: https://www.rfc-editor.org/rfc/rfc4511

[50] F. A. Ruambo, D. Zou, I. O. Lopes, B. Yuan, A. Muthanna, M. Zakarya, and M. S. A. Muthanna, “Trust scoring algorithms for zero trust-based software-defined perimeter architectures: A systematic literature review of advancements, challenges, and future directions,” Comput. Electr. Eng., vol. 132, Art. no. 111002, 2026, doi: 10.1016/j.compeleceng.2026.111002.